Name: Aastha Dhavan
SAP ID: 590015115
Batch: 2 [DevOps]

__/__/____

Object oriented Analysis and Design (CSEG3002)
Assignment-1

Handwritten Theory Assignment :-

① Explain basic notions of an object and class. Differentiate between an object and a class with a suitable real-world example.

Class & Object are two basic concepts. A class can be considered as a blueprint/template from which objects are created. It defines the properties (data) and behaviours (methods) that its objects will have.
An object is an actual instance of a class. It represents a particular real-world entity and has its own values for the properties defined in the class. An object can also perform the methods defined in its class. For eg: A Car class can have properties like Color, model, speed and methods start(), stop(), accelerate(). A particular car (Honda City) can be an object of the class Car.

Class	Object
• Blueprint / Template	• Instance of a class
• Defines properties & behaviours	• Contains actual values
• Logical concept	• Real entity from the class
• Can have many objects	• Belongs to a particular class
• Eg; Car	• Eg; my Car

② Explain encapsulation in OO development. How does encapsulation help in protecting an objects data and controlling access to its behaviour?

Encapsulation is one of the important of Object-oriented Programming. It means wrapping the data and methods that operate on data together inside a single class; restricting direct access to the internal data of an object.

__/__/____

It is commonly achieved using access modifiers such as private, public, protected. Imp: data members are generally kept private so that they cannot directly be accessed or changed from outside the class. Instead, controlled methods - getters and setters & other public methods can be used to access/modify data.
Eg; In a BankAcc class, balance should not directly be changed by anyone from outside the class. deposit(), withdraw() can be provided to control how balance is changed.
∴ It provides — • Data hiding
• Controlled Access
• Better Security
• Organized program and easier to maintain

③ Explain inheritance & polymorphism. Use a suitable example to show how inheritance supports reuse & how polymorphism allows the same oph to exhibit different behaviour.
Inheritance is a concept in which a child class acquires the properties & methods of a parent class. It mainly helps in code reusability because we can use existing code instead of writing it again. Foreg; an Animal class can have common methods eat(), sleep(). Classes like Dog, Cat can inherit these methods from Animal.
Polymorphism means many forms. It allows same method to behave differently from different objects. Foreg; sound() method can be present in Animal. A Dog can implement it as bark() and a Cat as meow().

Thus, inheritance helps in reusing code while polymorphism allows the same operation to work differently for different objects.

(4) Explain the importance of object-oriented concepts in software development. Discuss how encapsulation, inheritance and polymorphism contribute to modularity, reusability, maintainability & clarity.

Object-oriented concepts are important because they make software easier to develop, understand & maintain. They allow a large program to be divided into smaller classes and objects that represent real-world entities.

- Encapsulation keeps data & methods together inside a class and protects data from direct access. This improves modularity (seperate class) & security.
- Inheritance allows a new class to reuse the properties & methods of an existing class. This reduces duplicate code & improves reusability (existing code to be reused)
- Polymorphism allows same method to behave differently for different objects. This makes the program more flexible & easier to extend.

→ Maintainability by making changes easier to manage
→ Clarity because classes & objects represent real-world entities.

(5) A university library has different types of users such as Student and Faculty. Identify suitable classes and objects and explain where encapsulation, inheritance and polymorphism can be applied in this system.

For a university library system, a general class called LibraryUser can be created. It can contain common details such as userId, name and methods borrowBook() and returnBook().

The Student and Faculty classes can inherit from LibraryUser. They can use the common features of LibraryUser and can also have their own specific information. For eg; Student may have StudentId and Course while Faculty may have employeeId and department. A seperate Book class can also be created to represent books in the library.

Class	Purpose
LibraryUser	Parent class containing common user details
Student	Child class representing student users
Faculty	" " " faculty "
Book	Represents books available in the library

The relationship between LibraryUser and Student is inheritance, because a Student is a type of LibraryUser. Similarly, Faculty is also a type of LibraryUser.
The relationship between LibraryUser & Book is an association, because a user can borrow and return books. Thus, the structure can be represented as -

LibraryUser → Student
LibraryUser → faculty
LibraryUser ↔ Book

This design avoids repetition & makes the library system easier to manage and extends.